ImLosingIt Documentation

Created by

Sajakorn Hiranwipas 6731352921

Rahut Nopnirapath 6731345521

2110215 Programming Methodology

Semester 2 Year 2024

Chulalongkorn University

Virtual Maakratoo

Introduction

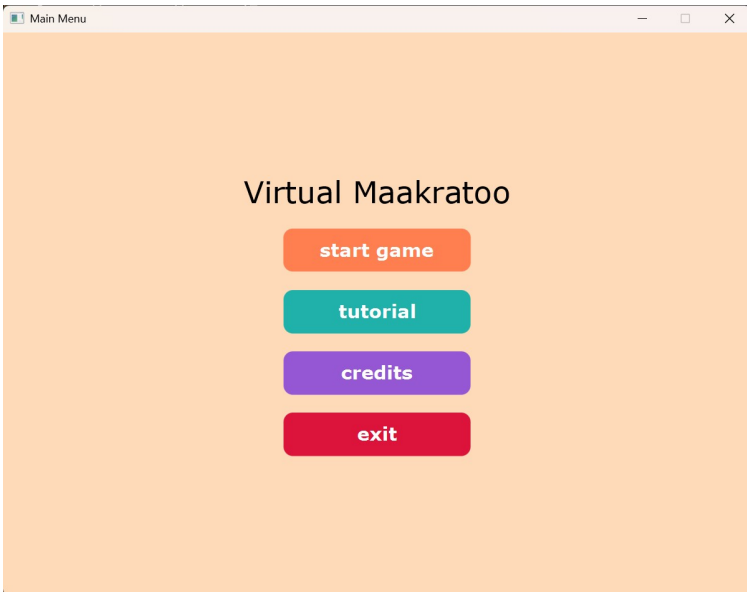
Virtual Maakratoo is a game made by 2 Mookrata-enthusiasts.
It is a game where you can enjoy the soothing vibe of Mookrata on your desk. The objective of the game is to have you chill and grill all the different types of meat, and try to not burn them! (Or if you are a hardcore gamer, there will be a timer which can be used to get some speedrun action going)

Rules

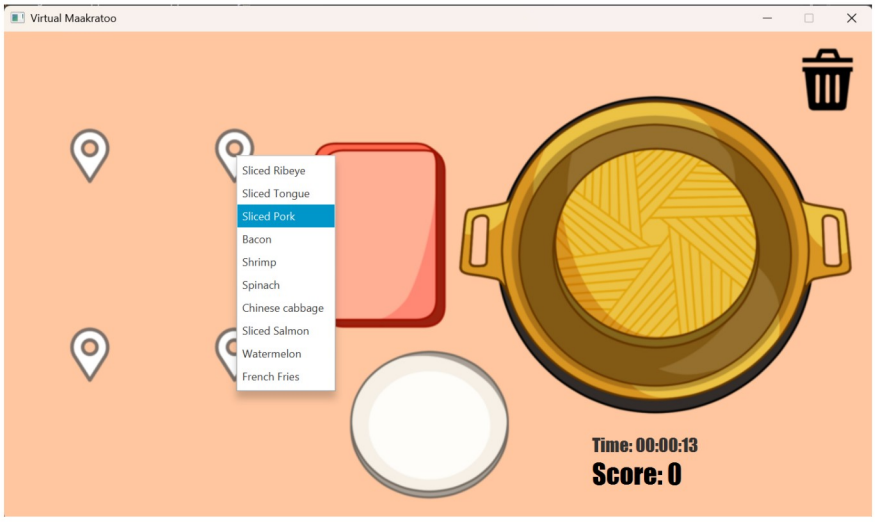
- You will gain points for each and every edible item you eat, and will lose points for every burnt item.

Example

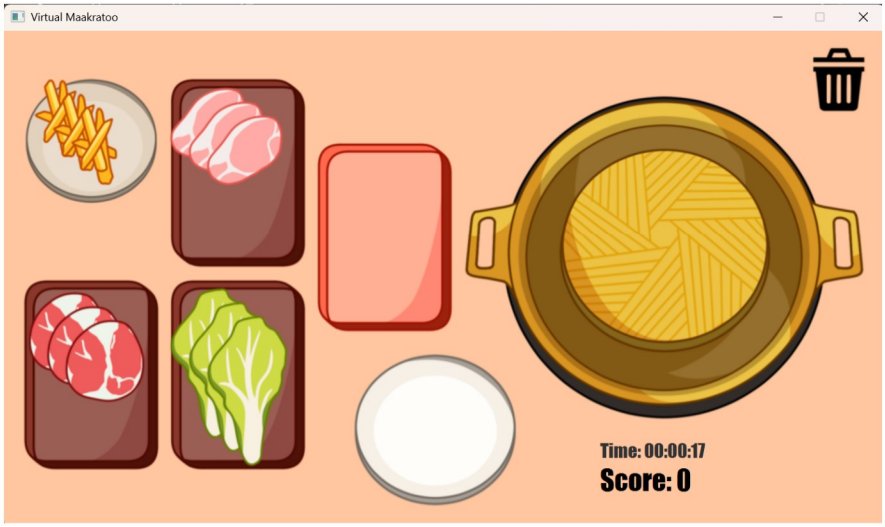
- To start the game click “start game” button, you will be met with our “Virtual Maakratoo” game.



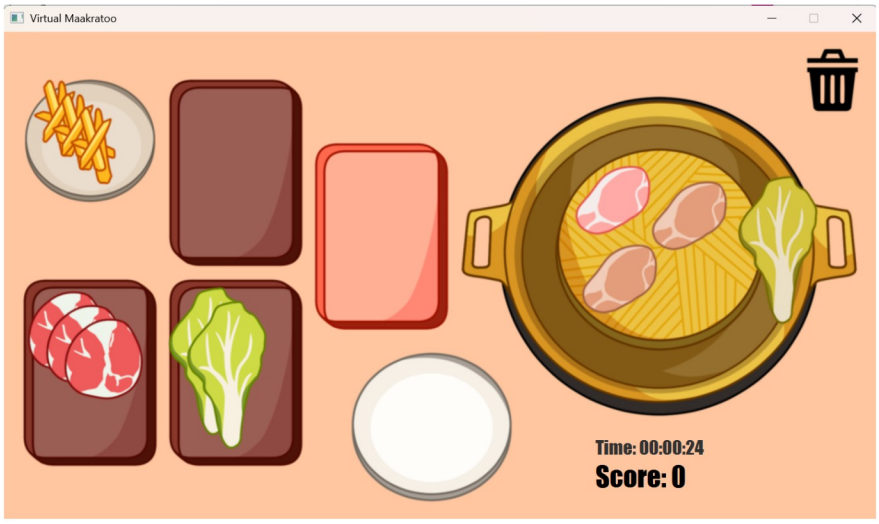
- Choose your items by clicking the “  ” icon.



- Since you are done picking out your favorite items, let’s get cooking!



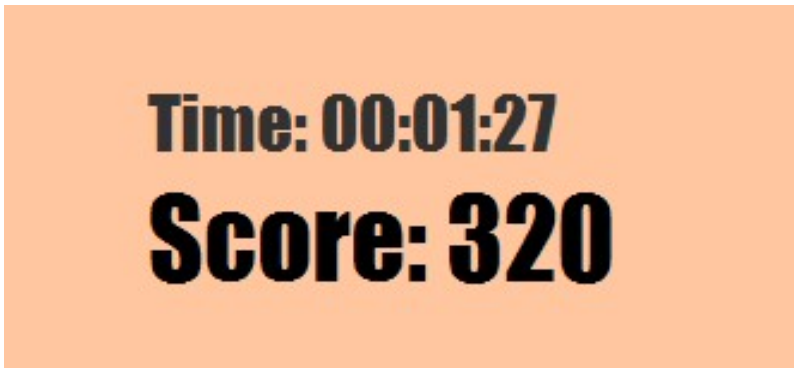
- Grab your items, put them on the grill, pay attention to the sizzling noise, It means they are ready to be flipped or eaten!



- Let's say you accidentally burn an item, that specific item will not be edible anymore, and we will not tolerate you trying to eat it, so we will be forcing you to throw it away by kindly dragging it to the “ 🗑 ”icon.

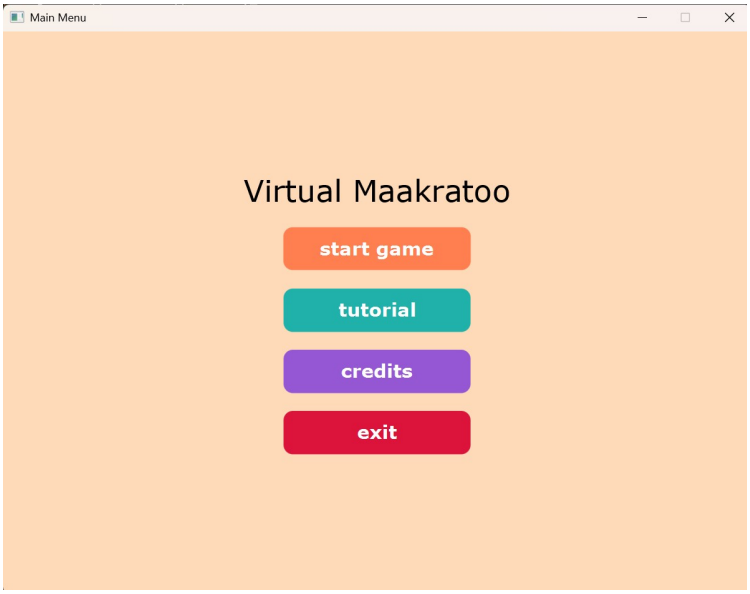


- After playing for a while, you can see your progress while the time is still continue counting.



MainMenu Scene

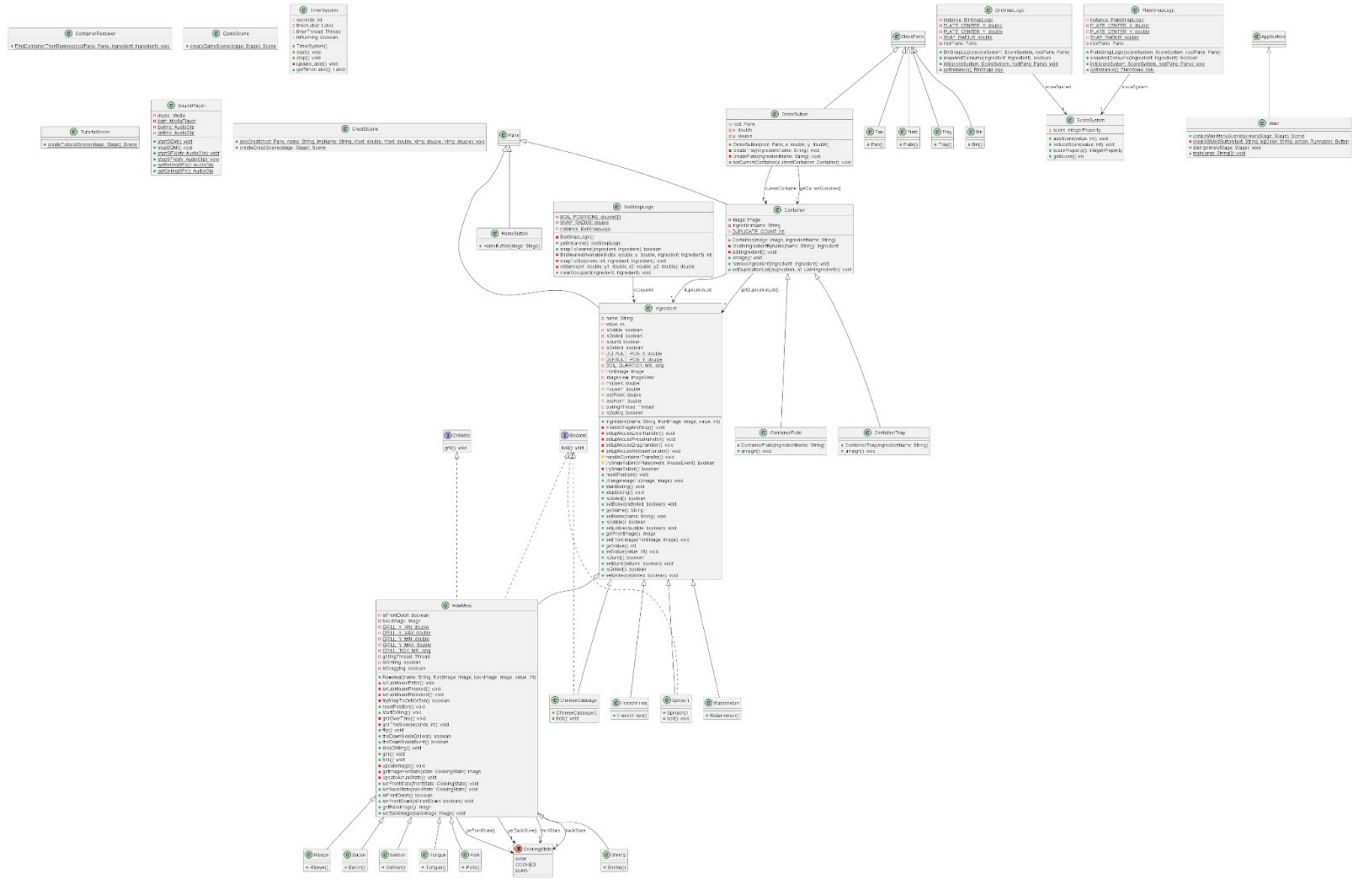
The tutorial button will tell you how to play, the credit button will show the game creators and the exit button to exit the game.



In the tutorial scene and credit scene, there's a home button on the top left corner. When pressed, you will be sent back to the main menu.



Class diagram



access modifier notations are listed below
+ (public)
(protected)
- (private)
Underline (static)
Italic (final)

1. Package base

1.1 interface Boilable

This defines an object that can be boiled.

+ void boil()	Boil the object.

1.2 interface Grillable

This defines an object that can be grilled.

+ void grill()	Grill the object.

1.3 enum CookingState

This shows the state of a grillable object.

+ enum CookingState {
RAW, COOKED, BURNT;
}
	State of grillable objects.

1.4 class Ingredient extends StackPane

Class that contains most methods meant to be extended.

1.4.1 fields

- String name	Name of the ingredient.
- int value	Value of the ingredient.
- boolean isEdible	A field denotes if the Ingredient is edible or not. Set its default as false.
- boolean isBoiled	A field denotes if the Ingredient is boiled or not. Set its default as false.
- boolean isBurnt	A field denotes if the Ingredient is burnt or not. Set its default as false.
- boolean isGrilled	A field denotes if the Ingredient is grilled or not. Set its default as false.
# double DEFAULT_POS_X	Default position of ingredient's center tray in the x-axis. Set it to 370.
# double DEFAULT_POS_Y	Default position of ingredient's center tray in the y-axis. Set it to 140.
- long BOIL_DURATION_MS	Boiling duration. Set it to 4000.
- Image frontImage	Ingredient's frontside image.
- ImageView imageView	Ingredient's imageview.
# double mouseX, mouseY	Position of mouse in x-axis and y-axis.
# double lastPosX, lastPosY	Latest position of ingredient in x-axis and y-axis.
- Thread boilingThread	A thread for counting the boiling duration.
- boolean isBoiling	A field denotes if the Ingredient is boiling or not

1.4.2 constructors

+ Ingredient(String name, Image frontImage, int value)	Initialize Fields. Then call enableDragAndDrop().

1.4.3 methods

- void enableDragAndDrop()	Enable all mouse setups.
- void setupMouseEnterHandler()	When hovering over an ingredient, change the cursor to HAND.
- void setupMousePressHandler()	When clicked on an ingredient, change the cursor to CLOSED_HAND. If the ingredient is currently boiling, stop it.
- void setupMouseDragHandler()	Sync ingredient position with current mouse position.
- void setupMouseReleaseHandler()	When released, change the cursor to DEFAULT. If this can't snap to any areas, reset its position by calling resetPosition().
- void handleContainerTransfer()	Removes the ingredient from that container then Adds it to the root pane of the scene and converts the ingredient's local position to scene coordinates.
# boolean trySnapToBinOrPlate(MouseEvent event)	If the ingredient is in the plate's or bin's snap radius, return true.
- boolean trySnapToBoil()	If the ingredient is boilable and is in the boil's snap radius, store the current position, call startBoiling(), return true. Else, Calls resetPosition() and returns false.
+ void resetPosition()	If it has already been boiled, go back to the last valid position. Else, call clearOccupant(this) from BoilSnapLogic

	and move it back to its default position.
+ void changeImageTo(Image image)	Changes the current image displayed for the ingredient.
+ void startBoiling()	If already boiled, return immediately. Else Sets isBoiling to true. Then starts a new background boiling thread.
+ void stopBoiling()	Stops any ongoing boiling process. Set isBoiling to false and interrupt boiling thread.
+ getters/setters	

1.5 class RawMeat extends Ingredient implements Grillable,Boilable

Base class of ingredient that are grillable/boilable

1.5.1 fields

- CookingState frontState	State of the frontside of the ingredient.
- CookingState backState	State of the backside of the ingredient.
- boolean isFrontDown	A field denotes if the frontside of the ingredient is down or not
- Image backImage	Ingredient's backside image.
- *double GRILL_X_MIN*	Minimum x-axis position for grilling area. Set it to 645.
- *double GRILL_X_MAX*	Maximum x-axis position for grilling area. Set it to 800.
- *double GRILL_Y_MIN*	Minimum y-axis position for grilling area. Set it to 150.

- *double GRILL_Y_MAX*	Maximum y-axis position for grilling area. Set it to 300.
- *double* GRILL_TICK_MS	Grilling duration per second. Set it to 1000.
- Thread grillingThread	A thread for counting the grilling duration.
- boolean isGrilling;	A field denotes if the Ingredient is grilling or not
- boolean isDragging	A field denotes if the Ingredient is dragged or not

1.5.2 constructors

+ RawMeat(String name, Image frontImage, Image backImage, int value)	Initialize fields.

1.5.3 methods

- void void setupMouseEnter()	When hovering over an ingredient, change the cursor to HAND.
- void setupMousePressed()	When clicked on an ingredient, change the cursor to CLOSED_HAND. ● Sync ingredient position with current mouse position. ● Stop boiling ● Stop grilling ● Set isDragging to true ● If pressing SPACE while isDragging , call flip()
- void setupMouseReleased()	When released, change the cursor to DEFAULT and set isDragging to false. If this can't snap to any areas, reset its position by calling resetPosition().
- boolean trySnapToGrillOrBoil()	If the ingredient is boilable and is in the grill's or boil's snap radius, store the

	current position, call startGrilling or startBoiling(), return true. Else, returns false.
+ void resetPosition()	If it has already been grilled or boiled, go back to the last valid position. Else, move it back to its default position.
+ void startGrilling()	Start grillingThread by calling grillOverTime.
- void grillOverTime()	If the downside is not cooked, call grillThisSide(4) and grill it. Then, if the downside is not burnt, call grillThisSide(4) and grill it again.
- private void grillThisSide(int seconds)	Print “tick” for GRILL_TICK_MS, for its respective seconds parameter.
+ void flip()	Flip this ingredient to another side then call changeImageTo() to flip the current image.
+ boolean theDownSideIsGrilled()	Check if the downside of the ingredient is grilled or not by checking its CookingState.
+ boolean theDownSideIsBurnt()	Check if the downside of the ingredient is burnt or not by checking its CookingState.
+ void stopGrilling()	Interrupt grilling thread.
+ void grill()	Set the CookingState of the downside of this ingredient from RAW to COOKED or from COOKED to BURNT, then call updateImage() and updateActualState().
+ void boil()	Change any RAW CookingState to COOKED. Then updateActualState()
- void updateImage()	Set both front image and back image to the correct image by calling getImageForState().
- Image getImageForState(CookingState	Return image that is related to its

state)	CookingState.
- void updateActualState()	If both sides are cooked, set isGrilled and isEdible to true. And if that ingredient is salmon, change its value to 100. Or if one or more sides are burnt, set isBurnt to true and set isEdible to false.
getters/setters	

2. Package gui

2.1 class Bin extends StackPane

Bin for disposing burnt ingredients

2.1.1 constructors

+ Bin()	Initialize image, imageView then add imageView.

2.2 class Pan extends StackPane

Pan for boiling/grilling ingredients

2.2.1 constructors

+ Pan()	Initialize image, imageView then add imageView.

2.3 class Plate extends StackPane

Plate for eating ingredients

2.3.1 constructors

+ Plate()	Initialize image, imageView then add imageView.

3. Package ingredient

3.1 class Bacon extends RawMeat

3.1.1 constructors

+ Bacon()	Initialize name, front view, back view and value.

3.2 class ChineseCabbage extends Ingredient implements Boilable

3.2.1 constructors

+ ChineseCabbage()	Initialize name, front view and value.

3.2.2 methods

+ void boil()	Boil the ingredient and change their image.

3.3 class FrenchFries extends Ingredient

3.3.1 constructors

+ FrenchFries()	Initialize name, front view and value. Set isEdible to true.

3.4 class Pork extends RawMeat

3.4.1 constructors

+ Pork()	Initialize name, front view, back view and value.

3.5 class Ribeye extends RawMeat

3.5.1 constructors

+ Ribeye()	Initialize name, front view, back view and value.

3.6 class Salmon extends RawMeat

3.6.1 constructors

+ Salmon()	Initialize name, front view and value. Set isEdible to true.

3.7 class Shrimp extends RawMeat

3.7.1 constructors

+ Shrimp()	Initialize name, front view, back view and value.

3.8 class Spinach extends Ingredient implements Boilable

3.8.1 constructors

+ Spinach()	Initialize name, front view and value.

3.8.2 methods

+ void boil()	Boil the ingredient and change their image.

3.9 class Tongue extends RawMeat

3.9.1 constructors

+ Tongue()	Initialize name, front view, back view and value.

3.10 class Watermelon extends Ingredient

3.10.1 constructors

+ Watermelon()	Initialize name, front view and value. Set isEdible to true.

4. Package logic

4.1 class BinSnapLogic

4.1.1 fields

- BinSnapLogic instance	Instance of itself.
- final double PLATE_CENTER_X	Horizontal bin's center. Set this to 900.
- final double PLATE_CENTER_Y	Vertical bin's center. Set this to 35.
- final double SNAP_RADIUS	Radius of bin that can snap ingredients. Set this to 70.
- final ScoreSystem scoreSystem;	Score system.
- Pane rootPane	Main container for all UI components in the current scene.

4.1.2 constructors

+ BinSnapLogic(Pane rootPane)	Initialize rootPane.

4.1.3 methods

+ boolean snapAndConsume(Ingredient ingredient)	If a burnt ingredient is within SNAP_RADIUS destroy it and reduce the score for the ingredient value.
+ void init(Pane rootPane)	Create a new instance of

	BinSnapLogic(Pane rootPane).
+ BinSnapLogic getInstance()	Return instance of itself.

4.2 class BoilSnapLogic

4.2.1 fields

- double[][] BOIL_POSITIONS	This field stores 2 positions that can be used for boiling. Those two positions are (590, 240) and (850, 240).
- double SNAP_RADIUS	Radius of boilable positions that can snap ingredients. Set this to 50.
- Ingredient[] occupants	This field stores ingredients that occupy those 2 positions that can be used for boiling.
- BoilSnapLogic instance	Instance of itself.

4.2.2 constructors

- BoilSnapLogic()	Initialize occupants size.

4.2.3 methods

+ BoilSnapLogic getInstance()	If this doesn't have an instance yet, create it.
+ boolean snapToNearest(Ingredient ingredient)	Call findNearestAvailableSlot and snapToSlot methods if possible.
- int findNearestAvailableSlot(double x, double y, Ingredient ingredient)	find the nearest available boil position.
- void snapToSlot(int index, Ingredient ingredient)	Move the ingredient to the nearest available boil position.
- double distance(double x1, double y1,	Return the distance between two points.

double x2, double y2)	
+ void clearOccupant(Ingredient ingredient)	Remove this ingredient from the occupant list.

4.3 class PlateSnapLogic

4.3.1 fields

- PlateSnapLogic instance	Instance of itself.
- double PLATE_CENTER_X	Position of plate's center in the x-axis. Set this to 460.
- double PLATE_CENTER_Y	Position of plate's center in the y-axis. Set this to 430.
- double SNAP_RADIUS	Radius of plate that can snap ingredients. Set this to 80.
- *ScoreSystem scoreSystem*	Score system.
- *Pane rootPane*	Main container for all UI components in the current scene.

4.3.2 constructors

+ PlateSnapLogic(ScoreSystem scoreSystem, Pane rootPane)	Initialize ScoreSystem and Pane.

4.3.3 methods

+ boolean snapAndConsume(Ingredient ingredient)	Snap and destroy any ingredient that enters the plate's SNAP_RADIUS. Then add the score for the ingredient value.
+ void init(ScoreSystem scoreSystem, Pane rootPane)	Create an instance of itself.
+ static PlateSnapLogic getInstance()	Return instance of itself.

4.4 class ScoreSystem

4.4.1 fields

- IntegerProperty score	Score for this game.

4.4.2 methods

+ void addScore(int value)	Increase score by value amount.
+ IntegerProperty scoreProperty()	Return score's property.
+ int getScore()	Return score.

4.5 class TimerSystem

4.5.1 fields

- int seconds	Playtime in second.
- *Label timerLabel*	Label timer.
- Thread timerThread	Thread for timer.
- boolean isRunning	Timer running status.

4.5.2 constructors

+ TimerSystem()	Initialize timerLabel.

4.5.3 methods

+ void start()	Set isRunning to true then start timerThread.
+ void stop()	Set isRunning to false and Stop timer.

	Reset time to 0 and update the label.
- void updateLabel()	Formats the seconds into hours:minutes:seconds and updates the label.
+ Label getTimerLabel()	Getter for TimerLabel.

5. Package main

5.1 class CreditScene

5.1.1 methods

+ void addCredit(Pane root, String name, String imgName, double Xtext, double Ytext, double Ximg, double Yimg)	Create credit with image and name, also set position in both x and y axis. Then add it to the pane.
+ Scene createCreditScene(Stage stage)	Initialize pane and scene. Then call addCredit(...) and add homeButton.

5.2 class GameScene

5.2.1 methods

+ Scene createGameScene(Stage stage)	Initialize pane, scene, bin, plate, pan, tray, scoreSystem, scoreLabel, timerSystem, order buttons then add all to pane.

5.3 class TutorialScene

5.3.1 methods

+ Scene createTutorialScene(Stage stage)	Initialize pane and scene. Add howToPlay.png and homeButton.

5.4 class Main

5.4.1 methods

+ Scene createMainMenuScene(Stage primaryStage)	Initialize main menu text, start button, tutorial button, credit button and exit button. Then create a VBox layout adding all components and add layout to the scene.
- Button createStyledButton(String text, String bgColor, Runnable action)	Helper to create customized buttons including new background, font, font size and button size .
+ void start(Stage primaryStage)	Initialize the stage then start BGM, setResizable(false) and show stage.
+ void main(String[] args)	Launch game.

6. Package util

6.1 class SoundPlayer

6.1.1 fields

- Media music	Load BGM.mp3 from the res folder then convert the URL to String.
- MediaPlayer bgm	Music player used to control background music.
- AudioClip boiling	Loads and prepares the sound effect for boiling
- AudioClip grilling	Loads and prepares the sound effect for grilling

6.1.2 methods

+ void startBGM()	Play background music in a loop.
+ void stopBGM()	Stop background music.

+ void startSFX(AudioClip sfx)	Play sound effects.
+ void stopSFX(AudioClip sfx)	Stop the sound effect.
+ getters/setters	

6.2 class ContainerRemover

6.2.1 methods

+ void FindContainerThenRemove(Pane rootPane, Ingredient ingredient)	Find the container that contains this ingredient. If successful, remove that ingredient from the container. If that container is now empty, remove it from the rootPane.

7. Package container

7.1 class abstract Container

7.1.1 fields

- Image image	An image for this container
- String ingredientName	Ingredient's name
# static final int DUPLICATE_COUNT	Size of duplicationList for creating duplicationList, set it to 3.
# List<Ingredient> duplicationList	List of ingredients that are contained in this container, Initialize.

7.1.2 constructors

+ Container(Image image, String ingredientName)	Initialize fields, call addIngredient() and arraign().

7.1.3 methods

- Ingredient createIngredientByName(String name)	Return a new ingredient by name.
- void addIngredient()	Add ingredients to duplicationList for DUPLICATE_COUNT.
+ abstract void arraign()	Arraign ingredients positions.
+ void removeIngredient(Ingredient ingredient)	Remove an ingredient from duplicationList and from this container.
+ getters/setters	

7.2 class ContainerPlate extends Container

7.2.1 constructors

+ ContainerPlate(String ingredientName)	Initialize name of the ingredient and image of the plate.

7.2.2 methods

+ void arraign()	Override the arraign method with proper positions.

7.3 class ContainerTray extends Container

7.3.1 constructors

+ ContainerTray(String ingredientName)	Initialize name and image of the ingredient.

7.3.2 methods

+ void arraign()	Override the arraign method with proper positions.